

IEEE SSCS Egypt Student Design Competition 2026

FPGA-Based Edge-AI Vision Accelerator

Team Abtal Eldigital

Ain Shams University - Faculty of Engineering
Computer and Systems Engineering

Team Members

Mina Hany Eid
Ziad Mostafa Abdelwahed
Jomana Abdelmohsen Abdelatty
Omar Sherif Hussien

Table of Contents

1) System Overview	1
2) Competition Requirements and Team Result	1
3) I/O Specification and Numerical Format	1
3.1 Input Image	1
3.2 Programmable Kernel	2
3.3 Output and Activation	2
4) Boundary Handling and Output Size	2
5) Architecture and Dataflow	2
5.1 Top-Level Structure	2
5.2 Control FSM	2
5.2.1 Binary State Encoding and Rationale	3
5.3 Line Buffers and Memory Organization	3
5.4 Window Generator	4
5.5 Nine Parallel Multipliers and DSP Mapping	4
5.6 Pipelined Adder Tree	4
6) Pipeline, Latency, and Throughput	4
6.1 Eight-Stage MAC Pipeline	4
6.2 Why Pipelining Is Used	5
6.3 Valid-Signal Alignment	5
6.4 Frame Fill and Throughput	5
7) Fixed-Point and Bit-Width Analysis	5
8) FPGA Implementation Results	5
8.1 FPGA Board	6
8.2 Timing	6
8.3 Resource Utilization	6
8.4 Power	7
8.5 Figure of Merit	7
9) Golden-Model and Testbench Verification	8
9.1 Golden Reference Model	8
9.2 RTL Testbench Method	8
9.3 QuestaSim Automation (run.do)	9
9.4 Simulation Waveform	10
10) Design Decisions and Tradeoffs	11
11) Conclusion	11

List of Figures

Figure 1. Top-level FPGA convolution accelerator block diagram from the implemented design.....	2
Figure 2. Control_FSM state diagram derived from the current RTL.....	3
Figure 3. Two-row line-buffer organization and 3x3 window formation.....	4
Figure 4. Eight-stage multiplier and adder-tree pipeline.....	5
Figure 5 : Operating Frequency on Vivado.....	6
Figure 6. Vivado Timing Report.....	6
Figure 7. Vivado resource-utilization result used by the report (235 LUTs, 279 FFs, 0 BRAMs, 9 DSPs).....	7
Figure 8. Vivado power-analysis summary.....	7
Figure 9. Resource denominator and normalized Figure of Merit calculation.....	8
Figure 10. run.do SAIF capture automation for the fifth randomized test case and active_processing.saif generation.....	9
Figure 11. QuestaSim waveform showing the top-level streaming handshake, control signals, FSM counters, and delayed valid-output generation.....	10
Figure 12. Transcript showing the success of the 9 tests and confirming of design functionality.....	10

1) System Overview

The accelerator is a streaming 3x3 CNN convolution engine. Pixels enter in row-major order through a valid/ready handshake. The data-routing subsystem delays previous rows, constructs a 3x3 spatial window, and forwards all nine pixels to a parallel multiply-accumulate datapath. The kernel coefficients are programmable at run time.

For each legal 3x3 window, the arithmetic operation is:

$$y(r,c) = \text{sum}(i=0..2) \text{sum}(j=0..2) k(i,j) * I(r+i,c+j)$$

The implemented ordering is CNN-style cross-correlation (no kernel flip). The raw result is accumulated as a signed 20-bit value. A combinational formatter then applies the optional ReLU bonus function and saturates positive results above 65535 to 16'hFFFF.

2) Competition Requirements and Team Result

The competition requires a minimum 32x32 single-channel input, unsigned fixed-point activations, a programmable N x N kernel with signed 8-bit coefficients, stride 1, at least 16-bit signed output precision, golden-model verification, and FPGA implementation results. The table below maps those requirements to the implemented design.

Parameter	Competition Specification	Team Result	Units	Comments
Input image size	Minimum 32x32	32x32 verification image; top-level dimensions are parameterized	pixels	Meets the minimum size
Input type	Grayscale / single-channel	Single 8-bit pixel stream	-	One feature-map channel
Input precision	Unsigned fixed-point; justify choice	8-bit unsigned, 0..255	bits	Natural grayscale range; compact storage and arithmetic
Kernel size	N x N, programmable/configurable	N=3 fixed hardware shape; nine coefficients programmable at run time	-	3x3 chosen for common CNN/edge filters
Kernel precision	8-bit signed	8-bit signed, -128..127	bits	Directly matches competition requirement
Stride	1	1	pixel	One-pixel window movement
Padding	Not mandated	No padding / valid	-	Produces only complete 3x3 windows
Output precision	Minimum 16-bit signed	Signed 20-bit raw MAC; optional ReLU stream is unsigned 16-bit	bits	Raw convolution preserves signed result; ReLU removes negative values
Activation	ReLU optional bonus	Implemented	-	Negative MAC values clamp to zero
Architecture type	Explain architecture	Streaming fully parallel 3x3 MAC with line buffers and pipelined adder tree	-	One valid window can enter the MAC each cycle
Multipliers / MACs	Report	9 parallel multipliers + one 9-tap adder tree	-	Mapped to 9 DSPs in supplied utilization result
Pipeline stages	Report	8 MAC stages	cycles	4 multiplier-side + 4 adder-tree stages
Latency	Report	8-cycle MAC pipeline after registered window-valid event	cycles	Frame startup also includes line/window fill
Throughput	Report	1 output pixel/cycle	pixel/cycle	30 valid outputs per row after fill
FPGA utilization	LUT/FF/DSP/BRAM	235 LUT, 279 FF, 9 DSP, 0 BRAM	-	From supplied Vivado result
Maximum Frequency	Report maximum frequency	289.9 MHz max clock frequency	MHz	3.45 ns period in supplied timing figure
Power estimate	Report	0.156 total on-chip power	W	0.051 W dynamic + 0.105 W static
Verification	Golden model + test cases + comparison	Python golden model; 9-vector RTL regression testbench; supplied simulation waveform, verified the design against 9 test cases	-	Golden-model self-test passes; waveform included in Section 9.4.
Figure of Merit	Report	0.00936 using throughput = 1 pixel/cycle	-	Power x (LUT + 50 DSP + 100 BRAM) denominator = 106.86

3) I/O Specification and Numerical Format

3.1 Input Image

- Resolution used for verification: 32 x 32 pixels.
- Image type: grayscale / single-channel feature map.
- Pixel type: unsigned 8-bit [7:0], range 0..255.
- Streaming handshake: a pixel is accepted only when pixel_valid && pixel_ready is true.

The 8-bit input precision is a deliberate efficiency choice. It exactly covers standard grayscale values from 0 to 255, halves storage compared with a 16-bit pixel representation, and keeps multiplier and line-buffer widths small while retaining the full source image intensity range.

3.2 Programmable Kernel

- Kernel shape: 3 x 3.
- Coefficient precision: signed 8-bit, range -128..127.
- Kernel port: signed [71:0], containing nine 8-bit coefficients.
- The kernel is programmable at run time; changing the 72-bit input changes the filter without resynthesizing the accelerator.

3.3 Output and Activation

- Raw convolution result: signed [19:0] MAC_out.
- Formatter output: unsigned [15:0] pixel_out after ReLU and positive saturation.
- If MAC_out < 0, pixel_out = 0.
- If $0 \leq \text{MAC_out} \leq 65535$, pixel_out = MAC_out[15:0].
- If MAC_out > 65535, pixel_out = 65535.

4) Boundary Handling and Output Size

The accelerator uses stride 1 and no padding. Therefore, a legal output is produced only when all nine pixels of the 3x3 window are inside the 32x32 image.

- Valid output rows: 0..29
- Valid output columns: 0..29
- Output dimensions: $(32 - 3 + 1) \times (32 - 3 + 1) = 30 \times 30$
- Total valid output pixels: 900

The Control_FSM asserts a window-valid event only when the accepted input coordinate satisfies row_count ≥ 2 and col_count ≥ 2 . This suppresses the incomplete horizontal windows at the beginning of each new row and guarantees that the MAC only processes complete 3x3 neighborhoods.

5) Architecture and Dataflow

5.1 Top-Level Structure

The current Top_accelerator integrates four main subsystems: Control_FSM, Data_Router, MAC_array, and data_formatter_and_ReLU. The RTL data path is Data_Router -> MAC_array -> formatter, while Control_FSM manages input acceptance and aligns pixel_out_valid with the pipelined MAC result.

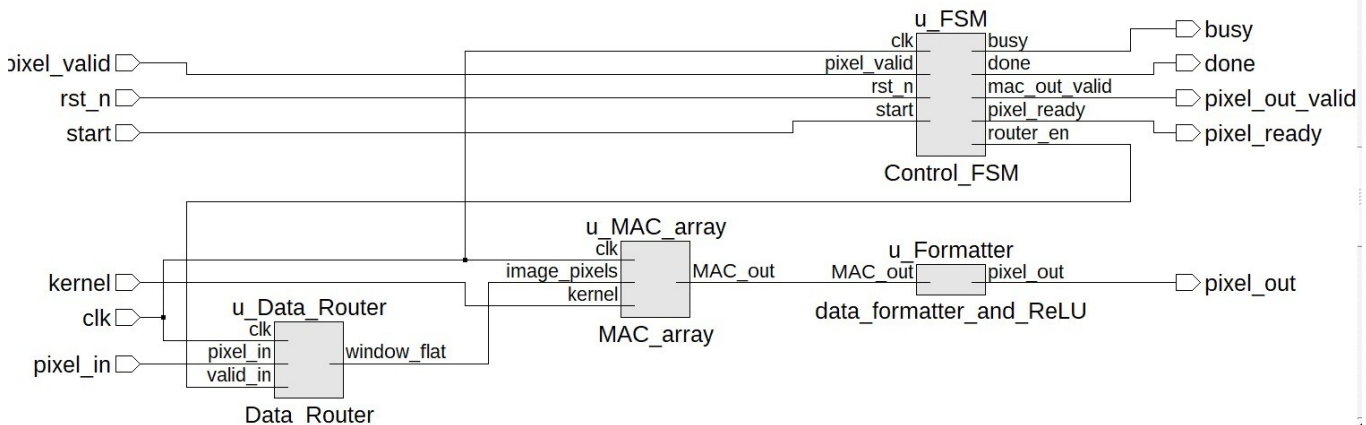


Figure 1. Top-level FPGA convolution accelerator block diagram from the implemented design.

5.2 Control FSM

The controller uses three states: IDLE, STREAM, and DRAIN. IDLE waits for start. STREAM accepts pixels and advances row/column counters only on a successful handshake. After the final input pixel is accepted, the FSM enters DRAIN, blocks additional input, and waits until the final valid MAC result exits the arithmetic pipeline. The done output is a one-cycle pulse aligned with the last valid result.

5.2.1 Binary State Encoding and Rationale

The implemented Control_FSM uses compact 2-bit binary state encoding in the RTL: ST_IDLE = 2'b00, ST_STREAM = 2'b01, and ST_DRAIN = 2'b10. The remaining code 2'b11 is unused; the FSM default branch returns to ST_IDLE, providing a defined recovery path for an illegal state code.

State	Binary Code	Role
IDLE	2'b00	Wait for start; controller not busy
STREAM	2'b01	Accept pixels and advance row/column counters
DRAIN	2'b10	Stop input and wait for the final pipelined output

Binary encoding was selected because only three control states are required. The theoretical minimum is $\text{ceil}(\log_2(3)) = 2$ state bits, so the implemented encoding uses two state flip-flops, whereas a one-hot implementation would require three. Fewer state registers reduce flip-flop utilization and slightly reduce clock load and state-register switching capacitance, which can reduce dynamic control power. The absolute saving is modest compared with the nine-DSP convolution datapath, but it keeps the controller compact and efficient.

Binary encoding also suits this controller because the decode logic is simple: pixel_ready is asserted only in STREAM, while busy only distinguishes IDLE from the two active states. One-hot encoding can be advantageous for larger or very timing-critical FSMs because it can simplify state decoding, but for this three-state controller it would add a state register with little practical benefit. The selected encoding therefore minimizes control overhead without creating a meaningful timing bottleneck.

The encoding is standard binary rather than Gray encoding: the STREAM (01) to DRAIN (10) transition changes two bits. Gray coding is not required here because all state transitions are synchronous and local to the FPGA clock domain; compact state storage is the more relevant objective for this small controller.

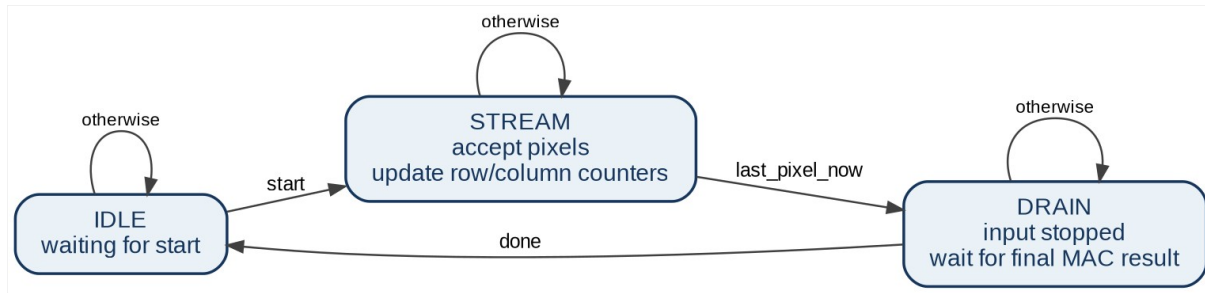


Figure 2. Control_FSM state diagram derived from the current RTL.

5.3 Line Buffers and Memory Organization

A 3x3 convolution requires three image rows to be available simultaneously. The live pixel stream already provides the newest row, so only the two previous rows must be stored. Data_Router therefore instantiates two Line_Buffer modules in cascade: LB1 delays by one complete image row and LB2 delays by two complete rows.

Each Line_Buffer is coded as a FIFO-style shift register. When valid_in is asserted, the new pixel is inserted at buffer[0], every stored element shifts by one position, and the oldest value at buffer[Image_Width-1] becomes delayed_pixel. For IMAGE_WIDTH = 32, each buffer stores only 32 x 8 = 256 bits; both buffers together store 512 bits.

For this small line length, a shift-register/LUT-oriented implementation is resource-efficient because dedicating a block RAM to only 256 bits per line would leave most of the memory block unused. The supplied Vivado utilization result reports 0 BRAMs, confirming that the implemented 32-wide design does not consume block RAM. The shift-register coding style also gives a direct one-sample-per-cycle delay with a simple oldest-sample tap.

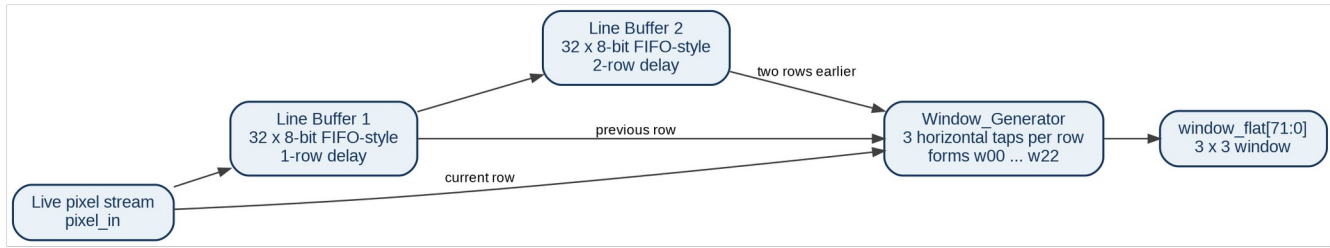


Figure 3. Two-row line-buffer organization and 3x3 window formation.

5.4 Window Generator

Window_Generator receives three synchronized row streams: LB2_out for the oldest row, LB1_out for the previous row, and pixel_in for the live row. Each row contains three horizontal shift registers. On every valid input, the registers shift and the newest column enters w02, w12, and w22. The resulting 3x3 window is packed as:

$$window_flat = \{w00, w01, w02, w10, w11, w12, w20, w21, w22\}$$

This fixed ordering is matched by the golden model and by the kernel packing used in the testbench.

5.5 Nine Parallel Multipliers and DSP Mapping

A single 3x3 window contains nine pixel-coefficient pairs. The design computes all nine products in parallel rather than time-multiplexing one multiplier over nine cycles. This is an explicit area-for-throughput tradeoff: it consumes nine DSP resources, but it allows a new complete window to enter the arithmetic pipeline every valid cycle, achieving Throughput of 1 output pixel/cycle.

parallel_multipliers.v uses a generate loop to create nine identical registered multiplier paths. Each path performs the following operations:

- Kernel byte: sign-extended from signed 8-bit to the 18-bit A input register.
- Pixel byte: zero-extended from unsigned 8-bit to the 25-bit B input register, so values 128..255 remain positive.
- Multiplication: performed in a registered signed product stage marked with (* use_dsp = "yes" *) to encourage DSP48E1 mapping.
- Additional product pipeline register followed by a registered 16-bit multiplier output.

The product range is -32640 to +32385, so every legal 8-bit pixel x signed 8-bit coefficient product fits exactly in signed 16 bits. Therefore taking the 16-bit product output does not discard valid numerical information for the specified input ranges.

5.6 Pipelined Adder Tree

The nine 16-bit products are accumulated with a balanced, registered adder tree instead of a serial chain. A serial nine-term addition would create a longer combinational path. The tree reduces the depth of combinational logic between registers and explicitly grows the signed width at each stage: 16-bit products -> 17-bit partial sums -> 18-bit sums -> 19-bit sums -> signed 20-bit MAC_out.

Each addition sign-extends its operands before summation, preserving the sign and preventing overflow at intermediate stages. The registered tree improves timing at the cost of additional flip-flops and latency.

6) Pipeline, Latency, and Throughput

6.1 Eight-Stage MAC Pipeline

The current Control_FSM documents and tracks an eight-stage arithmetic pipeline. Stages 1-4 are associated with the multiplier path and stages 5-8 are the registered adder tree:

Stage	Register / Operation	Purpose
1	A/B input registers	Register sign-extended kernel and zero-extended pixel
2	Parallel multiplication	Nine products computed concurrently
3	Product pipeline register	Breaks multiplier path for timing
4	16-bit product output register	Produces nine registered products
5	Adder stage 1	16-bit products -> 17-bit partial sums
6	Adder stage 2	17-bit -> 18-bit sums
7	Adder stage 3	18-bit -> 19-bit sums

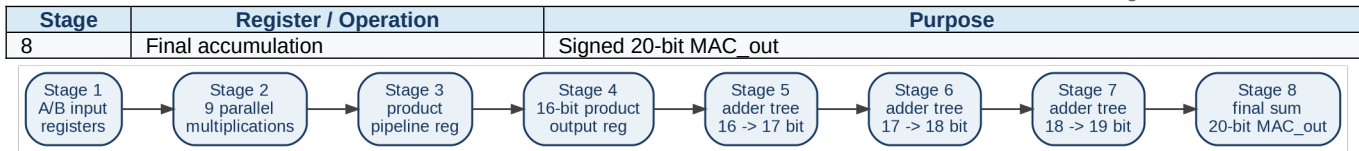


Figure 4. Eight-stage multiplier and adder-tree pipeline.

6.2 Why Pipelining Is Used

Pipelining divides a long arithmetic operation into shorter registered sections. This reduces the amount of combinational logic that must settle within a single clock period, allowing a higher operating frequency and a high steady-state throughput. Different windows can occupy different stages at the same time: while one window is being accumulated, following windows can already be in multiplication and input-register stages.

The tradeoff is increased latency and register count to achieve higher frequency and throughput = 1. One window needs several cycles to traverse the pipeline, but after the pipeline fills, one result can emerge for every cycle that corresponds to a legal input window.

6.3 Valid-Signal Alignment

Control_FSM shifts window_valid through an eight-bit valid_pipe. mac_out_valid is assigned from valid_pipe[7], aligning pixel_out_valid with the final registered MAC result. A parallel last_pipe performs the same delay for the final-window marker so that done is asserted with the last valid output.

6.4 Frame Fill and Throughput

The first legal 3x3 window is completed when input coordinate (row=2, col=2) is accepted, which is the 67th pixel of a 32-wide row-major frame. The corresponding arithmetic result then traverses the registered MAC pipeline. During each row, after the first two invalid horizontal positions, the design can produce one valid result per cycle for 30 consecutive positions.

- Peak / valid-window throughput: 1 output pixel per cycle.
- Valid outputs per 32x32 frame: 900.
- Input pixels per frame: 1024.

7) Fixed-Point and Bit-Width Analysis

The competition requires the report to explain numerical width, overflow, truncation, saturation, and rounding. The current design uses integer-valued fixed-point quantities with zero fractional bits. The selected widths are sufficient for the complete legal input range.

Quantity	Implemented Width	Range / Meaning	Handling
Input pixel	Unsigned 8-bit	0 .. 255	No conversion loss
Kernel coefficient	Signed 8-bit	-128 .. 127	Competition-required coefficient width
Single product	Signed 16-bit	-32640 .. 32385	Fits exactly; no product overflow
Adder stage 1	Signed 17-bit	Two-product partial sum	Operands sign-extended before add
Adder stage 2	Signed 18-bit	Up to four products	Explicit bit growth
Adder stage 3	Signed 19-bit	Up to eight products	Explicit bit growth
Raw MAC output	Signed 20-bit	-293760 .. 291465 worst case	Fits inside -524288 .. 524287
Formatted output	Unsigned 16-bit	0 .. 65535	ReLU + upper saturation

Worst-case positive sum = $9 \times 255 \times 127 = 291465$. Worst-case negative sum = $9 \times 255 \times (-128) = -293760$. Both fit safely within a signed 20-bit accumulator. Therefore the accumulation itself requires no saturation or rounding. Saturation occurs only in the output formatter: negative values are clipped by ReLU and positive values above 65535 are clamped to 65535.

8) FPGA Implementation Results

The following values are taken from the supplied Vivado implementation screenshots. The descriptive text uses measured implementation values rather than the earlier rough resource estimates.

Metric	Result	Interpretation
LUTs	235	General logic, control, adders, routing/shift-register logic
Flip-Flops	279	Pipeline, state, counters, and data registers
DSPs	9	Nine parallel multiplier paths
BRAMs	0	Small 32-pixel line buffers implemented without block RAM
Clock period	4.0 ns	Corresponds to 295.858 MHz in supplied timing figure

Metric	Result	Interpretation
Operating Clock Frequency	250 MHz	Clock shown by Vivado result
Total on-chip power	0.156 W	0.051 W dynamic + 0.105 W device static

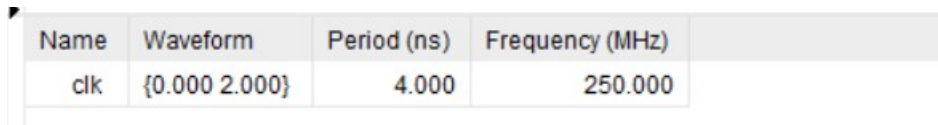
8.1 FPGA Board

The accelerator targets the AMD/Xilinx Zynq-7000 XC7Z020 SoC on the PYNQ-Z2 development board, utilizing exclusively its Artix-7 programmable logic fabric. The convolution datapath leverages LUTs, flip-flops, and DSP48E1 slices without depending on the dual-core ARM processing system. This platform was chosen for its widespread academic availability, robust Vivado toolchain support, and reliable standard interfaces. It provides substantial capacity headroom, comfortably accommodating the 9-DSP and 235-LUT implementation well below maximum device utilization limits.

8.2 Timing

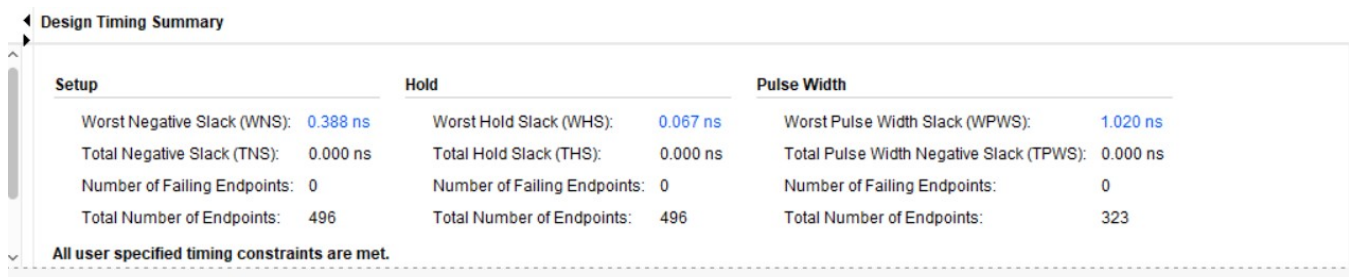
The timing summary demonstrates successful closure at a constrained clock period of 4.0 ns, corresponding to a operating frequency of 250.0 MHz.

- **Setup Timing:** Achieved a positive Worst Negative Slack (WNS) of +0.388 ns with a Total Negative Slack (TNS) of 0.000 ns and zero failing endpoints, proving the design meets all maximum-delay constraints.
- **Hold Timing:** Recorded a positive Worst Hold Slack (WHS) of +0.067 ns with zero failing endpoints, verifying that data stability is fully maintained across all registers.
- We chose operating frequency of 250 MHz provides the optimal engineering balance for the accelerator. It enables clean clock synthesis via standard PLL integer multipliers to avoid jitter, while maintaining a reliable 14% safety margin below the achieved F_{max} of 289.9 MHz to withstand thermal variations. Additionally, this frequency aligns seamlessly with standard AXI interconnect buses for smooth SoC integration and delivers an ideal power-throughput ratio that cuts dynamic power while comfortably supporting real-time Edge AI video processing.



Name	Waveform	Period (ns)	Frequency (MHz)
clk	{0.000 2.000}	4.000	250.000

Figure 5 : Operating Frequency on Vivado



Design Timing Summary		
Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.388 ns	Worst Hold Slack (WHS): 0.067 ns	Worst Pulse Width Slack (WPWS): 1.020 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 496	Total Number of Endpoints: 496	Total Number of Endpoints: 323
All user specified timing constraints are met.		

Figure 6. Vivado Timing Report.

8.3 Resource Utilization

The implementation uses 235 LUTs, 279 flip-flops, 9 DSPs, and 0 BRAMs. The 9-DSP count matches the architectural choice to compute all nine 3x3 products in parallel. The zero-BRAM result is consistent with the small FIFO-style line buffers being implemented in logic resources rather than dedicated block memory.



Figure 7. Vivado resource-utilization result used by the report (235 LUTs, 279 FFs, 0 BRAMs, 9 DSPs).

8.4 Power

Total on-chip power was evaluated at operating frequency of **250 MHz** using Vivado's post-implementation analysis. To achieve an accurate, design-specific measurement, a simulation-derived SAIF (Switching Activity Interchange Format) file was used rather than Vivado's default vectorless estimation.

To ensure the Figure of Merit (FOM) reflects true active efficiency, the power capture methodology was strictly controlled:

- **Active-Only Capture:** The SAIF capture window was gated exclusively to the busy signal. Isolating the active streaming and processing phases prevents reset and idle gaps from artificially diluting the average toggle rate and understating power.
- **High-Entropy Stimulus:** The simulation was driven specifically by the 05_random_image_random_kernel test vector. Unlike constant, monotonic, or corner-case vectors that under-stimulate the datapath, this randomized dataset provides high-entropy stimulus, realistically exercising the line buffers, nine-multiplier array, and adder tree.

Under these rigorous, active-operation conditions at 250 MHz, Vivado reports a total on-chip power of **0.156 W** (0.051 W dynamic; 0.105 W static). DSP and I/O blocks formed the largest dynamic power categories, consistent with the accelerator's streaming, computationally dense architecture.

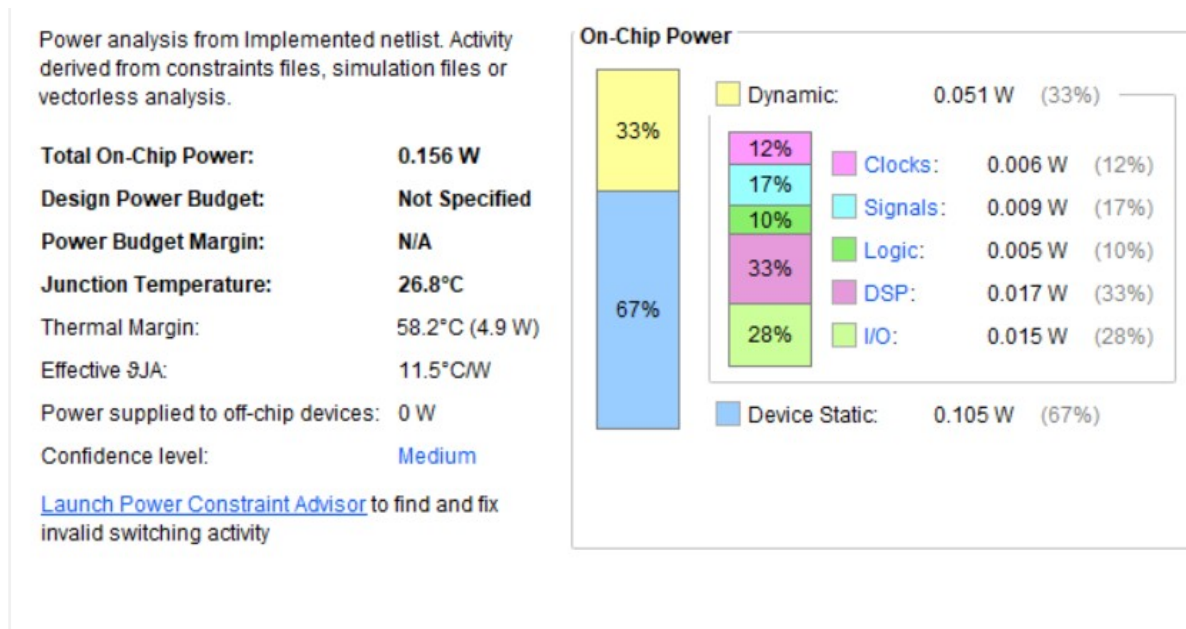


Figure 8. Vivado power-analysis summary.

8.5 Figure of Merit

The competition defines: $FOM = \text{Throughput} / [\text{Power} \times (\text{LUTs} + 50 \times \text{DSPs} + 100 \times \text{BRAMs})]$. Using the supplied implementation values and peak throughput = 1 output pixel/cycle:

- Resource term = $235 + 50 \times 9 + 100 \times 0 = 685$.
- Denominator = $0.156 \times 685 = 106.86$.
- $FOM = 1 / 106.86 = \text{approximately } 0.00936$.

Resource Denominator Calculation:

$$\text{Denominator} = \text{Total Power} \times (\text{LUTs} + 50 \times \text{DSPs} + 100 \times \text{BRAMs})$$

$$\text{Denominator} = 0.156 \times (235 + 50(9) + 100(0))$$

$$\text{Denominator} = 0.156 \times (235 + 450 + 0) = 0.156 \times 685 = 106.86$$

Normalized Figure of Merit (FOM) Calculation (Throughput = 1 pixel/cycle):

$$\text{FOM} = \frac{\text{Throughput}}{\text{Denominator}} = \frac{1}{106.86} \approx 0.00936$$

Figure 9. Resource denominator and normalized Figure of Merit calculation.

9) Golden-Model and Testbench Verification

9.1 Golden Reference Model

The software reference is implemented in Python and mirrors the current top-level numerical behavior: 32x32 unsigned 8-bit input, programmable signed 8-bit 3x3 kernel, stride 1, no padding, row-major cross-correlation, signed 20-bit raw accumulation, and ReLU plus positive saturation for the 16-bit output stream.

The model also matches the current byte packing. Window_Generator packs {w00, w01, w02, w10, w11, w12, w20, w21, w22}; the testbench packs the kernel in the corresponding row-major order. parallel_multipliers traverses the packed bytes from the least-significant slot, so matching image/kernel positions are multiplied together in reverse slot order and the final sum is unchanged.

The golden-model self-test was executed successfully. It confirms a 30x30 / 900-output shape, positive saturation of 291465 to 65535, negative ReLU of -293760 to 0, and the top-level packing probe value of 285.

9.2 RTL Testbench Method

Top_accelerator_tb.v reads input_image.txt, kernel.txt, and expected_output.txt for each vector. It drives the pixel stream only when pixel_ready is asserted and compares each pixel_out value whenever pixel_out_valid is high. Each case contains 1024 input pixels and 900 expected outputs. Nine cases therefore exercise 8100 formatted output comparisons.

Vector	Purpose	Key Expected Behavior
01_zero_image_identity	Zero image / identity kernel	All outputs 0
02_one_image_all_ones_kernel	Uniform image and accumulation	Every raw/formatted output = 9
03_increasing_image_identity	Raster order and identity response	Output follows valid center pixels
04_pattern_image_sobel_x	Signed edge kernel	Exercises positive/negative responses and ReLU
05_random_image_random_kernel	General stress case	Exercises mixed signed coefficients
06_max255_image_identity	Maximum input without saturation	Output = 255
07_max255_positive_saturation	Positive overflow handling	Raw = 291465, formatted = 65535
08_max255_negative_relu	Negative extreme	Raw = -293760, formatted = 0
09_top_packing_order_probe	Window/kernel byte-order check	First raw/formatted output = 285

9.3 QuestaSim Automation (run.do)

The project includes a QuestaSim/ModelSim automation script, run.do, so the complete RTL verification flow can be reproduced from one script instead of manually compiling modules and configuring the simulator each time. The script creates and maps the work library, compiles the RTL and Top_accelerator_tb.v in dependency order, and then launches work.Top_accelerator_tb with +acc signal visibility enabled.

- **Compilation and simulation setup:** vlib/vmap create the work library, vlog compiles data_formatter_and_ReLU, Control_FSM, Window_Generator, Line_Buffer, Data_Router, parallel_multipliers, adder_tree, MAC_array, Top_accelerator, and the testbench, and vsim -voptargs=+acc starts the integrated top-level simulation.
- **Waveform automation:** the script opens the wave, structure, and signals views and adds grouped traces for global controls, the input interface, the output interface, and internal FSM tracking. The monitored signals include clk, rst_n, start, busy, done, kernel, pixel_valid, pixel_ready, pixel_in, pixel_out_valid, pixel_out, state, row_count, and col_count. This is the signal set used for the waveform evidence in the following subsection.
- **Activity capture for power analysis:** the script counts frame-start events and enables recursive Questa power tracking only on the fifth test, 05_random_image_random_kernel. When that frame finishes, it writes active_processing.saif using power report -all -bsaif. Capturing the randomized active interval rather than reset/idle time gives Vivado switching activity that is representative of real accelerator operation.
- **Repeatability:** run -all lets the testbench execute the complete nine-vector regression automatically. Keeping compilation, waveform configuration, and SAIF generation in one run.do file reduces manual setup differences between simulation runs and makes the reported verification and power-measurement procedure reproducible.

```

51  # -----
52  # SAIF capture – arm ONLY on the 5th start/done pair
53  # (test 05_random_image_random_kernel, given tests run in
54  # 01..09 order in the testbench's initial block)
55  # -----
56  set start_count 0
57  set power_added 0
58  set power_reported 0
59  when {/Top_accelerator_tb/start == 1} {
60      incr start_count
61      if {$start_count == 5 && !$power_added} {
62          power add -r -in -inout -out -internal /Top_accelerator_tb/uut/*
63          set power_added 1
64          echo "Power tracking started at [now] (test 5, start #$start_count)"
65      }
66  }
67  when {/Top_accelerator_tb/done == 1} {
68      if {$power_added && !$power_reported} {
69          power report -all -bsaif active_processing.saif
70          set power_reported 1
71          echo "SAIF written at [now]"
72      }
73  }
74  run -all

```

Figure 10. run.do SAIF capture automation for the fifth randomized test case and active_processing.saif generation.

9.4 Simulation Waveform

The supplied QuestaSim waveform provides timing-level evidence for the integrated top-level design. It displays `clk`, `rst_n`, `start`, `busy`, `done`, the programmable kernel, `pixel_valid/pixel_ready/pixel_in`, `pixel_out_valid/pixel_out`, and the internal FSM state, `row_count`, and `col_count`. The row and column counters show row-major streaming progress, while the later `pixel_out_valid` assertion illustrates that a valid convolution result appears only after the line/window fill and the registered MAC pipeline delay.

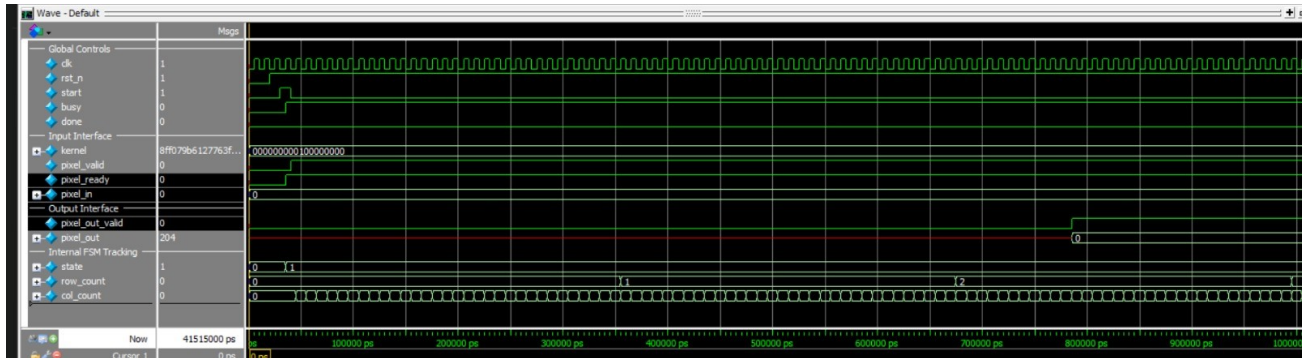


Figure 11. QuestaSim waveform showing the top-level streaming handshake, control signals, FSM counters, and delayed valid-output generation.

```
# Running test: cnn_golden_model_package_v2/golden_vectors/01_zero_image_identity
# Completed cnn_golden_model_package_v2/golden_vectors/01_zero_image_identity with 0 errors.
#
# Running test: cnn_golden_model_package_v2/golden_vectors/02_one_image_all_ones_kernel
# Completed cnn_golden_model_package_v2/golden_vectors/02_one_image_all_ones_kernel with 0 errors.
#
# Running test: cnn_golden_model_package_v2/golden_vectors/03_increasing_image_identity
# Completed cnn_golden_model_package_v2/golden_vectors/03_increasing_image_identity with 0 errors.
#
# Running test: cnn_golden_model_package_v2/golden_vectors/04_pattern_image_sobel_x
# Completed cnn_golden_model_package_v2/golden_vectors/04_pattern_image_sobel_x with 0 errors.
#
# Running test: cnn_golden_model_package_v2/golden_vectors/05_random_image_random_kernel
# Power tracking started at (test 5, start #5)
# SAIF written at
# Completed cnn_golden_model_package_v2/golden_vectors/05_random_image_random_kernel with 0 errors.
#
# Running test: cnn_golden_model_package_v2/golden_vectors/06_max255_image_identity
# Completed cnn_golden_model_package_v2/golden_vectors/06_max255_image_identity with 0 errors.
#
# Running test: cnn_golden_model_package_v2/golden_vectors/07_max255_positive_saturation
# Completed cnn_golden_model_package_v2/golden_vectors/07_max255_positive_saturation with 0 errors.
#
# Running test: cnn_golden_model_package_v2/golden_vectors/08_max255_negative_relu
# Completed cnn_golden_model_package_v2/golden_vectors/08_max255_negative_relu with 0 errors.
#
# Running test: cnn_golden_model_package_v2/golden_vectors/09_top_packing_order_probe
# Completed cnn_golden_model_package_v2/golden_vectors/09_top_packing_order_probe with 0 errors.
#
# =====
# ALL 9 TESTS PASSED SUCCESSFULLY!
# =====
```

Figure 12. Transcript showing the success of the 9 tests and confirming of design functionality

10) Design Decisions and Tradeoffs

Nine parallel multipliers vs. one reused multiplier: Nine concurrent products consume more DSP resources, but avoid a nine-cycle multiplication loop and enable one complete 3x3 window to enter the MAC every valid cycle.

Deep pipeline vs. single-cycle arithmetic: Eight registered MAC stages increase per-window latency and FF usage, but shorten the critical path, support a high clock rate, and allow many windows to be processed concurrently.

Shift-register line buffers vs. BRAM: According to competition scope which is Edge AI Accelerator for small feature map, Shift-register/LUT-based line buffers were chosen over BRAM to maximize resource efficiency for small feature maps within the competition's Edge-AI scope. Because coarse-grained BRAM primitives are vastly oversized for small buffers, using them would result in severe resource underutilization. While the shift-register architecture scales efficiently up to 128x128 pixels and remains functional up to 256x256.

No padding vs. same-size output: No padding simplifies boundary control and avoids storing or generating artificial border pixels, but reduces the 32x32 input to a 30x30 output.

20-bit raw MAC vs. narrower accumulator: The 20-bit width is selected from worst-case arithmetic bounds and prevents accumulation overflow across all legal 8-bit inputs and signed 8-bit coefficients.

ReLU/saturation post-processing: The optional bonus activation is combinational and adds no pipeline stage. It makes the top output non-negative and limits it to 16 bits, but the signed raw MAC must remain conceptually distinct from the activated output.

11) Conclusion

The implemented accelerator uses a streaming architecture tailored to the 32x32 competition image size. Two compact FIFO-style row delays and a 3x3 window generator feed nine parallel DSP multipliers, followed by a registered adder tree with explicit signed bit growth. The eight-stage MAC pipeline trades extra registers and latency for a high operating frequency and a peak throughput of one valid output per cycle. The supplied Vivado results report 235 LUTs, 279 FFs, 9 DSPs, 0 BRAMs, 250 MHz operating clock frequency, and 0.156 W total on-chip power, producing the reported FOM of approximately 0.00936.

The Python golden model is aligned with the current Top_accelerator packing and arithmetic and provides nine directed/random verification vectors. The current RTL testbench consumes those vectors and performs cycle-valid output comparisons. The supplied top-level waveform is included in Section 9.4 as timing evidence.